Computer-Aided Design 157 (2023) 103479

Contents lists available at ScienceDirect

Computer-Aided Design

journal homepage: www.elsevier.com/locate/cad

Recognizing geometric primitives in 3D point clouds of mechanical CAD objects

Chiara Romanengo [1], Andrea Raffo [1], Silvia Biasotti *, Bianca Falcidieno

Istituto di Matematica Applicata e Tecnologie Informatiche "E. Magenes", Consiglio Nazionale delle Ricerche, Via de Marini 6, 16149 Genova, Italy

ARTICLE INFO

Article history:
Received 5 August 2022
Received in revised form 10 November 2022
Accepted 10 January 2023

Keywords:
Geometric primitives
Point clouds
Mechanical CAD objects
Hough transform

ABSTRACT

The problem faced in this paper concerns the recognition of simple and complex geometric primitives in point clouds resulting from scans of mechanical CAD objects. A large number of points, the presence of noise, outliers, missing or redundant parts and uneven distribution are the main problems to be addressed to meet this need. In this article we propose a solution, based on the Hough transform, that can recognize simple and complex geometric primitives and is robust to noise, outliers, and missing parts. Additionally, we can extract a series of geometric descriptors that uniquely characterize a primitive and, based on them, aggregate the output into maximal or compound primitives, thus reducing oversegmentation. The results presented in the paper demonstrate the robustness of the method and its competitiveness with respect to other solutions proposed in the literature.

© 2023 Elsevier Ltd. All rights reserved.

1. Introduction

The increasing availability of affordable digital scanning devices – such as close-range photogrammetry and laser scanners – has made point clouds one of the most common ways of representing the surface of an object. In Computer-Aided Design, this fact results in the need to extract information from measured data points using higher-level geometric primitives. To give an example, it is highly convenient to recognize and reconstruct a digital model so that it can be interpreted in terms of some basic components and easily manipulated by CAD systems. A large number of points, the presence of noise and outliers, the occurrence of missing or redundant parts and the non-uniform distribution of the data severely limit the use of tessellations (e.g., meshes) as a means to ease the analysis and reconstruction of shapes; rather, they make it more convenient to analyse the point cloud directly [1].

Many approaches for detecting primitives lack robustness to noise and outliers or deal only with mesh models. Most of them are able to extract only a few classes of simple geometric primitives, being planes and cylinders the most commonly recognized, and do not consider more complex basic shapes, such as surfaces of revolution or generalized cylinders. Moreover, many methods have a tendency to oversegment, thus producing very fragmented output without aggregating the non-contiguous subsets of points that lie on the same primitive. The detection of maximal or compound primitives (e.g. cylinders composed of non-adjacent parts, or patterns) is another open problem, mostly addressed only for planes or cylinders [2,3]. For example, if a pipe is interrupted by another part, as in the block model in Fig. 5(b), traditional methods recognize the two parts of the pipe as two distinct primitives. This is particularly important, for instance, when decomposing patterns that correspond to scans of assembly CAD models, because it concurs with the recognition of compound primitives and patterns [4]. When it comes to memory usage and computational complexity, learning approaches require onerous training and large annotated databases; on the other hand, the direct detection of mathematical primitives in a general space embedding typically faces the computational limitation of dealing with a number of free parameters that rapidly grows as the complexity of the primitives increases.

In this paper, we deal with the recognition of simple and complex geometric primitives in point clouds originating from scans of mechanical CAD objects, where recognition means the detection of the primitives in a given point cloud and the extraction/computation of the best (geometric) parameters associated with those primitives. We adopt an approach based on the Hough transform (HT) to do this. The HT meets the needs of being able to recognize multiple instances of primitive functions and, through a voting procedure, to be robust to noise and outliers. Originally limited to straight lines in images [5], it has been generalized and extended in multiple directions to handle other shapes, also using families of non-parametric templates in images [6] or point

* Corresponding author.
E-mail addresses: chiara.romanengo@ge.imati.cnr.it (C. Romanengo), andrea.raffo@ge.imati.cnr.it (A. Raffo), silvia@ge.imati.cnr.it (S. Biasotti), bianca@ge.imati.cnr.it (B. Falcidieno).
[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.cad.2023.103479
0010-4485/© 2023 Elsevier Ltd. All rights reserved.

clouds from CAD objects [7]; recently, the extended version proposed in [8] has opened up the HT to a wide range of algebraic primitives.

Here we generalize the HT to surface primitives – not necessarily algebraic – represented in parametric form. Our solution is tailored to deal with point clouds and is able to deal with simple geometric primitives and some complex ones (generalized cylinders and cones, surfaces of revolution, helical surfaces, etc.), including maximal and compound primitives. To this end, we aggregate the primitives found on the basis of their size and space embedding, whenever possible. Our strategy also reduces the oversegmentation of the output and we exploit the strategy proposed in [9] to decrease the computational complexity of methods based on the HT thanks to an opportune preprocessing of the point cloud and its subparts. Our experiments confirm the robustness and completeness of the method, and the comparisons made exhibit its competitiveness with respect to the methods proposed so far in the literature. To sum up, our main contributions are:

- The introduction of a geometric primitive recognition method that is particularly robust to noise and outliers and is able to recognize multiple instances of the same primitive.
- The extraction and recognition of complex primitives, in addition to the most common ones (planes, cylinders, cones, spheres, tori).
- The aggregation of primitives on the basis of their shape, size and position to detect maximal and compound primitives.
- An extensive experimentation on different datasets and comparison with state-of-the-art methods.

The rest of the paper is organized as follows. Section 2 overviews previous work in geometric primitives detection and HT-based recognition methods. Section 3 briefly recalls the basic concepts of HT. Section 4 introduces and lists the geometric primitives that can be identified by our method, with emphasis on complex primitives such as general cylinders and cones, surfaces of revolutions, helical surfaces, and convex combinations of curves. Section 5 describes our approach for the identification of primitives in point clouds based on the HT, and the search for maximal primitives and possible relationships between primitives through a clustering technique. Section 6 provides experimental results of our method for the identification of simple and complex primitives and shows a qualitative and quantitative comparative analysis. Concluding remarks end the paper.

2. Previous work

Representing an object with a set of geometric components is a long-standing problem in computer vision, computer graphics and CAD. As we are interested in recognizing (simple or complex) geometric primitives in the manufacturing domain, here we focus our attention on those approaches that share the same goal.

A considerable variety of algorithms have been devised to decompose digitalized point clouds or meshes representing CAD objects into regions approximated by primitives belonging to some given sets. According to [10], these approaches can be grouped into four families: stochastic, parameter space, clustering and learning techniques. The first group includes the RANSAC method [11] and its optimizations. The second family includes Hough-like voting methods and parameter space clustering, e.g., [12]. The third class gathers all the other clustering techniques, and can be classified into three main types: primitive-driven region growing, e.g., [13]; automatic clustering and Lloyd-based algorithms, e.g., [14]; primitive-oblivious segmentation, e.g., [15]. Finally, with the growing popularity of deep learning techniques, supervised fitting methods have been proposed even for multi-class primitives [16,17]. The reader is referred to [10] for a comprehensive historical taxonomy of methods for simple primitive detection, which is beyond the scope of this paper.

Despite a large number of available solutions, the problem is far from being solved; novel approaches have been proposed in the very last few years to address the shortcomings of existing paradigms or to propose novel directions to move in. A recent approach to deal with the recognition of geometric primitives is described in [18]. It consists of a curvature-based partitioning method that decomposes an input triangle mesh into maximal surface portions; the decomposition is performed so that each segment corresponds to one of the following seven invariance classes of surfaces: plane, sphere, cylinder, surface of revolution, prismatic, helix, and complex surface. The method was adapted to handle point clouds in [19]. In [20], a method for the identification and fitting of planes and cylinders from unstructured point clouds in manufacturing is presented. It consists of three subsequent phases: point cloud segmentation; merging of oversegmented regions and estimation of surface parameters; extraction of cylinders and planes. Being the method able to handle only two primitive types, its applicability is however restricted. To handle the increasing availability of acquired data, [1] introduces a region-growing-based system for the segmentation of large point clouds in planar regions. Other approaches, devised to detect only specific types of primitives, are: [21], which deals with quadric surfaces; [22] which fits surfaces of revolution; and [23], which extracts cylindrical shapes from non-oriented point clouds. In the field of deep learning, two methods have shown promising results in the problems of segmentation and recognition: ParSeNet [24] and HPNet [25]. Beyond their performance, outstanding merit is their capability of handling – together with the more classical simple geometric primitives – open and closed spline surfaces. Another learning-based method lately proposed for fitting primitives is PriFit [26], which learns to decompose a point cloud of various 3D shape categories into a set of geometric primitives, such as ellipsoids and cuboids, or alternatively deformed planes. However, the natural flexibility of supervised learning approaches in identifying geometric primitives comes with a considerable cost: the need of having gigantic labelled training sets, whose difficulty of gathering limits their current application.

As for the methods that use the HT to identify geometric primitives, they were limited, until recently, to planes [12,27], combinations of linear subspaces [28], spheres [29] and circular cylinders [30]; in the case of cylinders, however, the rotational axis needs to be known before the application of the Hough transform. The recent advances in the use of the algebraic functions [8] are paving the road to a larger use of the HT for recognizing more complex families of geometric primitives. To the best of our knowledge, the sole approaches that exploit this idea were proposed in [31], for the recognition of an ellipsoid in a free-form model; and in [9], for the recognition of spheres, (circular) cylinders, (circular) cones and tori in presegmented point clouds, as a way to post process faulty segmentations. The work in [9] also addresses the problem of reducing the number of the parameters of the primitive representation when the point cloud corresponds to a patch of a primitive: in this case, the point cloud is opportunely rototranslated in a space embedding so that it can be fit by a primitive in a standard form. The method has been recently compared with other direct and learning-based approaches, see [32]. However, the point clouds used for this comparison did neither originate from scans of mechanical CAD objects nor can be assumed to represent presegmented data, making the benchmark itself not of interest to this paper.

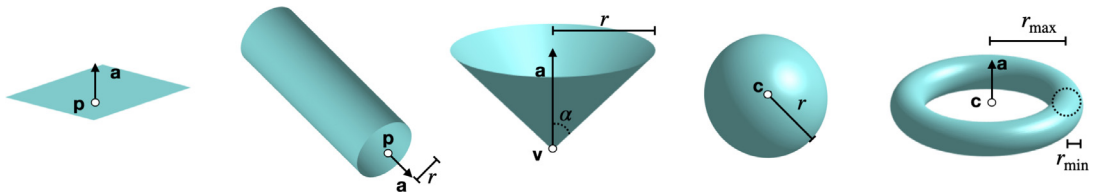


Fig. 1. Simple geometric primitives: plane, cylinder, cone, sphere and torus respectively, along with their attributes (geometric descriptors).

3. Basic concepts on the Hough transform

We denote by $\mathbb{A}^3_{\mathbf{x}}(\mathbb{R})$ and $\mathbb{A}^n_{\mathbf{A}}(\mathbb{R})$, respectively, the 3-dimensional and the n-dimensional affine spaces over $\mathbb{R}$, where $\mathbf{x} := (x, y, z)$ and $\mathbf{A} := (A_1, \ldots, A_n)$ vectors of indeterminates. Given a surface $\mathcal{S}$ defined as the zero locus of a function f, a parameter-dependent family of surfaces can be described by functions $f_{\mathbf{a}}$ as $\mathcal{F} = \{\mathcal{S}_{\mathbf{a}} : f_{\mathbf{a}}(x) = 0 \mid \mathbf{a} \in \mathcal{U}\}$, where $\mathcal{U}$ is an open set of the parameter space $\mathbb{A}^n_{\mathbf{A}}(\mathbb{R})$ and $\mathbf{a} := (a_1, \ldots, a_n)$ is the parameter vector. Then, given a point $P \in \mathbb{A}^3_{\mathbf{x}}(\mathbb{R})$, the HT of the point P with respect to the family $\mathcal{F}$ is $\Gamma_P = \{f_{\mathbf{a}}(P) = 0\} \subset \mathbb{A}^n_{\mathbf{A}}(\mathbb{R})$. For sufficiently general points $P \in \mathbb{A}^3_{\mathbf{x}}(\mathbb{R})$, it turns out that Γ_P are hypersurfaces in $\mathbb{A}^n_{\mathbf{A}}(\mathbb{R})$. If the set of hypersurfaces Γ_P generated by varying P on a given surface meets in one and only one point $\bar{\mathbf{a}} \in \mathcal{U}$, the family of surfaces $\mathcal{F}$ is called *Hough-regular*; the intersection point $\bar{\mathbf{a}}$ of the Hough transforms Γ_P uniquely identifies the surface $\mathcal{S}_{\bar{\mathbf{a}}}$ from which the points $P \in \mathbb{A}^3_{\mathbf{x}}(\mathbb{R})$ were sampled in the first place. If the HT regularity is not guaranteed, the solution to the intersection problem might be non-unique and a solution must be selected among more potential parameters solutions.

In the discrete scenario, the HT framework deals with the problem of finding a surface – within a family $\mathcal{F}$ of surfaces – that best approximates a particular shape, given in the form of a data set of N points $\mathcal{P}$, where $N \gg n$. The common strategy to identify the solution (or a solution), introduced in [8], is based on the so-called *accumulator function*; it consists in a voting system whereby each point in $\mathcal{P}$ votes a n-uple $\mathbf{a} = (a_1, \ldots, a_n)$; the most voted n-uple corresponds to the most representative surface for the profile.

While the HT traditionally deals with one shape at a time, the framework we propose in this paper can be directly applied to point clouds containing multiple shapes. The family of surfaces we consider is represented in parametric form and we assume that the system defining $\mathcal{S}_{\mathbf{a}}$ with respect to the Cartesian coordinates x, y, and z can be analytically solved with respect to the parameter vector $\mathbf{a} = (a_1, \ldots, a_n)$. A more detailed description of how this works in our case is given in Section 5.

4. Definition of complex geometric primitives

In addition to the simple geometric primitives of Fig. 1 (plane, cylinder, cone, sphere, and torus) – see [9] for their recognition in presegmented point clouds – in this section, we introduce a set of complex geometric primitives that to the best of our knowledge have never been used before for recognition by HT.

In this paper we express surfaces parametrically with respect to the variables u and v. For the sake of simplicity, some of the surfaces are here presented in their standard form or with respect to some specific axes; nevertheless, one can easily generalize these equations by applying a generic transformation of the special orthogonal group SO(3).

- *General cylinders*. A cylinder is a surface traced by a straight line of fixed direction, the *generatrix*, while moving along a curve, the *directrix*. Given a curve $(x(u), y(u), z(u)) := (f_1(\mathbf{a}, u), f_2(\mathbf{a}, u), f_3(\mathbf{a}, u))$ and a direction (l, m, n), the parametric representation of the corresponding cylinder is given by:

$$\begin{cases} x = f_1(\mathbf{a}, u) + lv \\ y = f_2(\mathbf{a}, u) + mv \\ z = f_3(\mathbf{a}, u) + nv. \end{cases}$$

  Note that a general cylinder depends on the parameters defining generatrix and directrix. The dictionary of curves to be considered as directrix is extremely rich, see [33]. Table 1(a) considers a 5−convexity curve as directrix, whose equation can be found in [33].
- *General cones*. A cone is a surface traced by a straight line, the *generatrix*, while gliding along a curve, the *directrix*, and passing through a fixed point, the *vertex*. Given a parametrized curve $(x(u), y(u), z(u)) := (f_1(\mathbf{a}, u), f_2(\mathbf{a}, u), f_3(\mathbf{a}, u))$ and a point $V = (x_V, y_V, z_V)$, the parametric representation of the corresponding cone is given by:

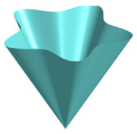
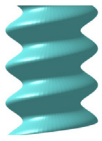
$$\begin{cases} x = x_V + (f_1(\mathbf{a}, u) - x_V)v \\ y = y_V + (f_2(\mathbf{a}, u) - y_V)v \\ z = z_V + (f_3(\mathbf{a}, u) - z_V)v. \end{cases}$$

  As in the case of cylinders, we can exploit the dictionary of plane curves to create families of cones. Then, a general cone depends on the parameters that define the curve and the components of the vertex.
  Table 1(b) shows a cone generated by a 5−convexity curve.
- *Surfaces of revolution*. A family of surfaces of revolution can be created by rotating a family of curves around an axis of rotation. For example, given the family of plane curves $(x(u), y(u), z(u)) := (f_1(\mathbf{a}, u), 0, f_2(\mathbf{a}, u))$ and the z-axis, we obtain the parametric equations

$$\begin{cases} x = f_1(\mathbf{a}, u) \cos v \\ y = f_1(\mathbf{a}, u) \sin v \\ z = f_2(\mathbf{a}, u). \end{cases}$$

  One of the most known examples is the *torus*; another example is given by ellipsoids, which are obtained by rotating an ellipse with respect to one of its principal axes. Table 1(c) shows the case of a surface obtained by rotating the curve $(x(u), y(u), z(u)) := (au, 0, b/u^5)$ around the z-axis.
- *Helical surfaces*. Table 1(d) presents a family of equations obtained by modifying the parametrization of a circular cylinder. Precisely, the radius R here varies between $[R_1, R_2]$, where $R_1 > 0$, by a cosine function; when radii are fixed, the slope of the output surface is controlled by the parameters in $z(u, v)$. Note that the radius variation can be adapted to other shapes (e.g., the triangle wave function).
- *Convex combination of curves*. It is possible to define surface primitives by considering the convex combination of a pair of parametrized curves $(f_1(\mathbf{a}, u), f_2(\mathbf{a}, u), f_3(\mathbf{a}, u))$ and $(g_1(\mathbf{b}, u), g_2(\mathbf{b}, u), g_3(\mathbf{b}, u))$. This family has the following

Table 1
Parametric representation of some complex geometric primitives.

(a) generalized cylinder	(b) generalized cone	(c) surface of revolution	(d) helical surface	(e) helical strip
				
$\begin{cases} x = \frac{a\cos u}{1+b\cos(5u)} \\ y = \frac{a\sin u}{1+b\cos(5u)} \\ z = v \end{cases}$	$\begin{cases} x = \frac{av\cos u}{1+b\cos(5u)} \\ y = \frac{av\sin u}{1+b\cos(5u)} \\ z = Av + B \end{cases}$	$\begin{cases} x = au\cos v \\ y = au\sin v \\ z = \frac{b}{u^5} \end{cases}$	$\begin{cases} x = R(u)\cos v \\ y = R(u)\sin v \\ z = k_1(u+nv) + k_2 \end{cases}$, where $R(u) := R_1 + \frac{R_2 - R_1}{2}(\cos u + 1)$, $n \in \mathbb{Z}$	$\begin{cases} x = R(u)\cos v \\ y = R(u)\sin v \\ z = v \end{cases}$, where R(u):=au+(1-u)b

parametric equations:

$$\begin{cases} x = vf_1(\mathbf{a}, u) + (1-v)g_1(\mathbf{b}, u) \\ y = vf_2(\mathbf{a}, u) + (1-v)g_2(\mathbf{b}, u) \\ z = vf_3(\mathbf{a}, u) + (1-v)g_3(\mathbf{b}, u), \end{cases}$$

where $v \in [0, 1]$. Note that the primitive parametrization depends on the same parameters which define the pair of curves, i.e., **a** and **b**. A planar example is given by the *annulus*, i.e., the region bounded by two concentric circles. A helical strip can be obtained by cutting and bending an annular strip; this corresponds to considering a convex combination of two helices of an equal slope but different radii. An example of a helical strip is provided in Table 1(e).

The use of the standard form allows us to diminish the number of unknown parameters in the recognition process. To give a practical example, the family

$$\begin{cases} x = a_1\cos(u) + b_1\sin(u) + c_1v + d_1 \\ y = a_2\cos(u) + b_2\sin(u) + c_2v + d_2 \\ z = a_3\cos(u) + b_3\sin(u) + c_3v + d_3, \end{cases}$$

contains (circular) cylinders; it has 12 unknown parameters, which make the representation computationally and memory-wise unmanageable when used in a voting procedure. By further assuming that the cylinder can be rototranslated so that it has the rotational axis aligned with the z-axis and it is centred in the origin, the number of parameters reduces to 1 in the case of a circular cylinder, i.e., the sole radius:

$$\begin{cases} x = r\cos(u) \\ y = r\sin(u) \\ z = v. \end{cases}$$

A strategy to obtain the standard form in an automatic way starting from a point cloud is provided in [9], and it can be naturally extended to complex primitives.

5. Recognizing geometric primitives using Hough transforms

In this section, we describe a method to recognize an input point cloud $\mathcal{P}$ with geometric primitives via the HT technique. It consists of the following main steps: an initial point cloud preprocessing followed by the iteration of a recognition step and a splitting phase, and a final step which applies a clustering technique to discover geometric relationships between primitives or parts of them. A graphical illustration of its pipeline is given in Fig. 2.

5.1. Point cloud preprocessing

First, the input point cloud $\mathcal{P}$ is translated to place its barycentre at the origin of the Cartesian coordinate system. Then, the normals at the input points are estimated. In the literature, there are several ways to estimate normals; in general, we are interested in using an approach suitable for handling noisy input.

For our purposes the orientation of the normal vector is not relevant, as we are only interested in aligning the most voted normal direction to the z-axis. We can therefore use the approach introduced in [9], which estimates the normal vectors through a voting procedure. For convenience, when $\mathcal{P}$ is clean we can also consider the method presented in [34], which is available in MATLAB with the `pcnormals` function; for each input point in $\mathcal{P}$, the method selects the k points of $\mathcal{P}$ closest to that input point and then uses principal component analysis (PCA) to estimate the normal vector. Note that this does not necessarily generate normal vectors that are coherent in their orientation; however, this is not a problem for our pipeline since we are not interested in the normal orientation but rather in the direction of the (estimated) normal vectors.

The point cloud is then rotated so that the most voted direction among the (just-estimated) normal vectors coincides with the z-axis. This is an essential step for multiple reasons. Firstly, our dictionary contains primitives that – in their standard form – are given with the axis coincident with the z-axis. In addition, this process is necessary to test the existence – as a first check – of primitives in their standard form (i.e., with centres or vertices in the origin of the Cartesian axes and with the normal or the principal axis aligned to the z-axis), thus reducing the number of parameters that will need to be estimated by the HT.

Finally, $\mathcal{P}$ is scaled into a unit cube, in order to restrict the range of variation for most parameters.

5.2. Recognition step

After being preprocessed, $\mathcal{P}$ becomes the input of the HT-based recognition step. Since $\mathcal{P}$ can contain different instances of the same primitive type and primitives of different types, the standard HT procedure must be adjusted to allow such cases. This step can be summarized as follows:

- *Selection of one or more families of primitives*. The user can select the families of primitives to be used for recognition; in its default configuration, the algorithm tests the presence of simple geometric primitives — one family at a time. By studying the geometric properties of $\mathcal{P}$ and by relating them to the geometric characteristics of the selected family

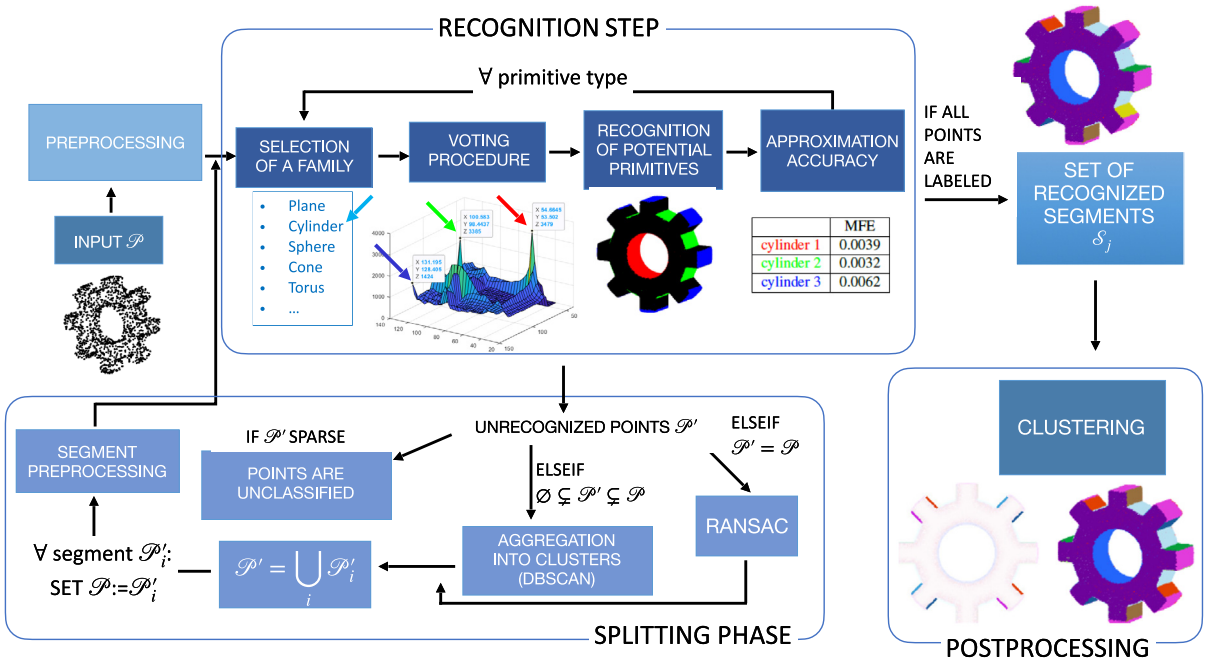


Fig. 2. Pipeline of the method.

(e.g., bounding box, radius, diagonal length), it is possible to initialize a region T of the parameter space. The region T is discretized into cells, which are uniquely identified by the coordinates of their centre. Then, an accumulator function H, in the discretized form of a matrix, is initialized. The matrix entries are in a one-to-one correspondence with the cells of the discretization performed in the previous step.

- *Estimation of the accumulator function.* An entry of the accumulator function H is increased by 1 each time the Hough transform Γ_P of a point P intersects the corresponding cell. The way we check if and where a Hough transform intersects some cells depends on whether the family of surfaces is described in implicit or parametric form. In our case, surfaces are expressed in parametric form; we adapt to surfaces the method devised for curves in [35]. Specifically, if the system of equations $f_{\mathbf{a}}$ defining the family can be solved analytically with respect to the unknown parameters $\mathbf{a}$, we automatically calculate a sample of Γ_P by exploiting the Moore–Penrose pseudo-inverse [36] of the matrix that defines the coefficients of $\mathbf{a}$. The evaluation of the intersection is translated into an inequality between the components of the sample points of Γ_P and the coordinates of the cell endpoints.
- *Selection of potential fitting primitives.* The cells corresponding to the peak values of the accumulator function H have to be identified. When the set of points is assumed to represent a single surface profile, the traditional HT formulation aims at finding the maximum of the accumulator function H; when the family of surfaces is not Hough-regular, there might exist several maxima. On the other hand, if the point cloud is composed of different primitives, different peaks of H identify potential primitives that might fit different parts. To select these peaks, we observe that peaks of the accumulator function corresponding to primitive shapes rise distinctly with respect to their neighbours and are well-characterized as isolated peaks. Formally said, we proceed by identifying peaks that have high *topological persistence*.[2] In our implementation, peaks that correspond to primitives are automatically recognized by keeping the local maxima having a persistence higher than 10% of the maximum value of H, by using the algorithm for persistent maxima proposed in [39]. The coordinates of the cell centres of the maxima correspond to the parameters of potentially recognized surface primitives.
- *Evaluation of the approximation accuracy.* To measure the recognition accuracy of a specific primitive, we select the set of input points $\mathcal{X}$ closer to such a primitive than a given threshold and study its density via the k-Nearest Neighbour algorithm (see, for example, [40]). If $\mathcal{X}$ is sparse, the recognized primitive is considered a false positive; otherwise, for each dense subset $\mathcal{X}_i \subset \mathcal{X}$ we define the *Mean Fitting Error* (MFE) as:

$$E(\mathcal{X}_i, \mathcal{P}) := \frac{1}{|\mathcal{X}_i|} \sum_{\mathbf{x} \in \mathcal{X}_i} d(\mathbf{x}, \mathcal{P})/l_i, \tag{1}$$

where $\mathcal{P}$ is the current primitive, d is the Euclidean distance, and l_i is the diagonal of the axis-aligned bounding box containing $\mathcal{X}_i$. What can happen when the selected family of primitives does not fit any part of the point cloud? There are two possibilities:

  - The accumulator function is identically zero, with the result that its persistence is zero; therefore the selection of potential fitting primitives returns the empty solution.
  - The accumulator function H does not present predominant peaks, resulting in false positives which are identifiable by studying the sparsity of the fitted points.

[2] The notion of topological persistence was introduced in [37] for encoding and simplifying the points of a filtration f by classifying them as either a feature or noise depending on its lifetime or persistence within the filtration. In practice, given a pair of points p and q, their persistence is defined as $f(p) - f(q)$. Pairing is defined in terms of the topological connection between the points, for details on topological persistence and saliency, we refer to [37,38]. In our case, the domain is represented by the grid of T, the role of filtration is played by the accumulator function H, and we are interested only in the peaks of H.

Given a set of dense points $\mathcal{X}_i$ and two candidate primitives $\mathcal{P}_{i,1}$ and $\mathcal{P}_{i,2}$, we first calculate the fitting errors $E(\mathcal{X}_i, \mathcal{P}_{i,1})$ and $E_i(\mathcal{X}_i, \mathcal{P}_{i,2})$ between each primitive and $\mathcal{X}_i$; the primitive having lowest error is kept.
Each time a primitive is recognized, the points of $\mathcal{P}$ close to the recognized primitive less than a threshold ε are discarded from $\mathcal{P}$. The value of ε typically ranges from 1% to 3% of the diagonal of the minimum bounding box of the $\mathcal{P}$, according to the type of primitive.

The algorithm returns the parameters of the geometric primitives and the corresponding points fitted by them, as well as the set of points that were not fitted by any primitive – denoted by $\mathcal{P}'$. Note that, if more geometric primitives potentially fit the same region of the point cloud, we select the one with the minimum fitting accuracy (i.e., the lowest MFE).

5.3. Splitting phase

The algorithm chooses the next step according to the resulting $\mathcal{P}'$:

- If $\mathcal{P}'$ is sparse, then its points are returned as unclassified.
- If $\emptyset \subsetneq \mathcal{P}' \subsetneq \mathcal{P}$, i.e., if $\mathcal{P}'$ is a proper nonempty subset of $\mathcal{P}$, we proceed by aggregating points in $\mathcal{P}'$ into clusters $\mathcal{P}'_j$, with $j = 1, \ldots, N_{\text{clust}}$, by adopting the *Density-Based Spatial Clustering of Applications with Noise* (DBSCAN) method [41], which groups together nearby points and marks as outliers isolated points in low-density regions. It requires two parameters: the first is a threshold used as the radius of the density region, while the second represents the minimum number of points required to form a dense region. Then, the recognition step from Section 5.2 is iterated over each cluster $\mathcal{P}'_j$ as long as some geometric primitives are recognized. Before proceeding with a new recognition round, we preprocess each cluster $\mathcal{P}'_j$; the preprocessing adopted here differs from that applied to the entire point cloud $\mathcal{P}$: we exploit the strategy presented in [9] to estimate its standard form, thus reducing the number of parameters that have to be estimated in the new iteration of the recognition step. Specifically, it computes the normals of the points of the cluster $\mathcal{P}'_j$ through a voting procedure and it estimates the position of vertices or centres and the direction of axes of symmetry, exploiting different geometric rules specialized for each type of primitive. To reduce the dimension of the parameter space, vertices and centres are translated to the origin of the coordinate system, while axes are aligned to the $z-axis$. Then, it automatically centres and orients a (spherical, cylindrical, conical or toric) cluster so that it can be fitted with a primitive in standard form. In case the recognition step involves complex primitives, the strategy proposed in [9] can be naturally extended since they are characterized by the presence of symmetry axes that can be estimated in a similar way through the use of the normals of points.
- It is possible that, in some steps, no primitive is recognized. This can happen in two cases: (i) when applying our algorithm to an input point cloud that has no primitives in their standard form, or (ii) when the estimation of the standard position of some cluster $\mathcal{P}'_j$ fails — because $\mathcal{P}'_j$ contains more than one primitive. In both cases, we proceed by oversegmenting the model: in our experiments, we use the RANSAC algorithm proposed in [11], because of its efficiency and its tendency to oversegment point clouds (see, for example, [15]); however, we proceed by estimating primitive types and geometric parameters with a new round of the recognition step, as voting procedures have shown greater robustness to point cloud artefacts.

This step ends by transforming the parameters found by the Hough transform with respect to the inverse translations, rotations and scaling applied in the previous steps of the algorithm. The result of this procedure is the decomposition of an input point cloud $\mathcal{P}$ into several subsets, called *surface segments* or simply *segments*, in such a way that points of the same segment are well approximated by the same primitive. We denote these final segments by $\mathcal{S}_j$.

5.4. Segment association based on geometric relationships

After decomposing the input point cloud into the segments $\mathcal{S}_j$, a clustering technique is applied to unveil geometric relationships. The goal of this step is to find maximal primitives that are not automatically detected in the HT-based recognition process, to detect patterns of primitives or to relate primitives (or parts of them) even if they do not belong to the same primitive. More specifically, we relate primitives on the basis of their positions, orientation and dimension, as described in [9] for simple geometric primitives: segments are aggregated only if they are part of the same primitives; otherwise, we only identify relationships that may be of interest, for example in the processing phases (e.g., process planning, machining). To give some examples:

- Segments sampled from circular cylinders can be associated with respect to their radii and rotational axes. For instance, it is possible to check whether such segments: originate from the exact same primitive shape, i.e., if they are all sampled from a cylinder of a given radius and rotational axis (such as for the red segments in Fig. 5); are characterized by the same radius and have parallel rotational axes (such as for the black segments in Fig. 8) ; share the radius but not the rotational axis, not even in terms of parallelism.
- Segments sampled from helical surfaces can be associated with respect to their parameters R_1, R_2 and n, k_1, k_2 and the direction — or a combination of such geometric descriptors.
- Segments sampled from n-convexity cylinders can be associated with respect to their parameters a and b, the number of convexities n, and the direction of the generatrix — or a combination of such geometric descriptors.

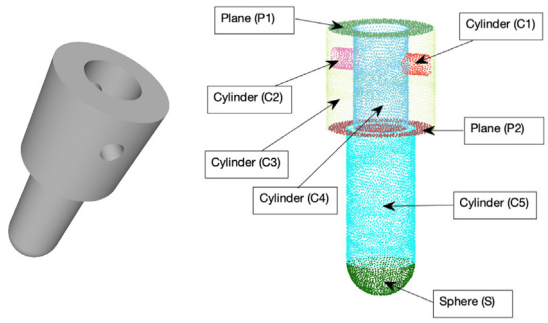
To perform segment associations, a well-known (hierarchical) clustering strategy is considered – the complete linkage – as it penalizes chaining effects.

5.5. An illustrative example

Fig. 2 provides a graphical illustration of the pipeline of our method. The main steps of the algorithm are associated with an example of a point cloud representing a gear.

After preprocessing the point cloud, we start with the first round of the recognition step — which aims at recognizing the presence of primitives in their canonical position. For the sake of clarity, however, the graphical illustration only focuses on the recognition of cylinders. Specifically, once the family of cylinders is selected, the corresponding accumulator function is computed; it exhibits three peaks, which indicate the presence of three potential solutions: the three cylinders highlighted in the colours red, green, and blue. The approximation accuracy for this type of primitive is less than 1%.

Being some segments (the non-axis-aligned planar segments) non in their canonical position, their points are not recognized in the first round of the recognition step. Instead, these points are collected in $\mathcal{P}'$ and aggregated into dense clusters in the splitting phase; each of such clusters is individually studied by a new round of the recognition step and recognized as planar segments. However, being these segments studied separately, we lose some



(a) (b)

equation	code	ground truth	HT parameters
$x^2+y^2=r^2$	C1	$r=1.50$	$r=1.50$
	C2	$r=8.00$	$r=8.00$
	C3	$r=4.00$	$r=4.00$
	C4	$r=1.50$	$r=1.50$
	C5	$r=5.00$	$r=5.00$
$z=k$	P1	$k=20.00$	$k=20.00$
	P2	$k=35.00$	$k=35.00$
$x^2+y^2+z^2=r^2$	S	$r=5.00$	$r=5.05$

(c)

Fig. 3. In (a) a mechanical CAD model from the benchmark in [42]; in (b) the vertices of its triangle mesh decomposed into 8 surface segments; in (c), for each primitive, the HT parameters are compared with those provided by the database.

geometric information. We then apply clustering to recognize that some planar segments lie on the same parametric plane, by exploiting the geometric descriptors provided by the HT.

The final result is a segmentation of 17 segments. It is worth mentioning that the method is able to group some of the non-adjacent parts, which belong to the same primitive in canonical position — as in the example of the two external cylinders, and of the axis-aligned planes that form four couples. However, for primitives not in their canonical position, postprocessing based on clustering is required.

5.6. Computational complexity

In the preprocessing step, our method includes the estimation of the normal vectors and the individuation of the most voted normal, which is implemented as explained in [34] – in the case of clean point clouds – and in [9] – in the presence of point cloud artefacts. For a thorough analysis of the computational complexities of this estimation, the reader is referred to the corresponding papers.

The formulation of the HT for surface primitives embedded in $\mathbb{R}^3$ naturally extends that for plane curves in $\mathbb{R}^2$. In the pipeline presented in this paper, for each point cloud $\mathcal{P}$ (both in the case of the initial point cloud and of single clusters at subsequent iterations) we apply the HT by considering the different types of geometric primitives one at a time; thus, the dimension of the parameter space changes accordingly to the type of primitive considered. The quantization of a region of interest and the dimension of the parameter space determines the size of the accumulator function, and dominate both the memory usage and the computational complexity of the Hough transform: it is, therefore, necessary to balance the samples for each parameter and their number. To reduce the computational cost, primitives are put in their (estimated) standard positions, in this way the number of parameters does not exceed 3, as explained in [9]; complementarily, adaptive approaches can be used to further speed up the search (e.g., [43]). The overall computational complexity of the HT-based recognition step is $\mathcal{O}(ML)$, where M denotes the number of cells of the parameter space and L represents the number of points (in the initial point cloud or in a cluster obtained at a subsequent iteration) at which we evaluate the HT accumulator function. More precisely:

- At the first iteration the HT is applied to the whole input point cloud. Supposing that the number of families of primitives to be used for the recognition is K and the number of points in $\mathcal{P}$ is N, the overall complexity of the first iteration is $\mathcal{O}(N\sum_{k=1}^{K} M_k)$, where M_k is the dimension of the parameter space for the kth primitive type.
- In the subsequent steps, the HT-based recognition method is applied to each cluster $\mathcal{P}'_j$ and then the computational cost corresponds to $\mathcal{O}(\sum_{j=1}^{N_{\text{clust}}} N_j \sum_{k=1}^{K} M_k)$, where N_j denotes the number of points in cluster $\mathcal{P}'_j$ while N_{clust} is the number of clusters.

Notice that, being the recognition step performed independently w.r.t. the primitive type, the task is *embarrassingly parallel*. The same applies to the clusters of points obtained in the same iteration.

Finally, agglomerative hierarchical clustering approaches require, in their naive implementation, $\mathcal{O}(N_{\text{seg}}^3)$ operations, where N_{seg} denotes the number of segments in the output segmentation; see, for example, [44]. When it comes to complete-linkage clustering, one can consider more efficient implementations, such as the one proposed in [45], which costs $\mathcal{O}(N_{\text{seg}}^2)$. Although the dissimilarity-matrix assembly costs $\mathcal{O}(N_{\text{seg}}^2)$, one may note that each entry is computed independently; again, the task is therefore embarrassingly parallel.

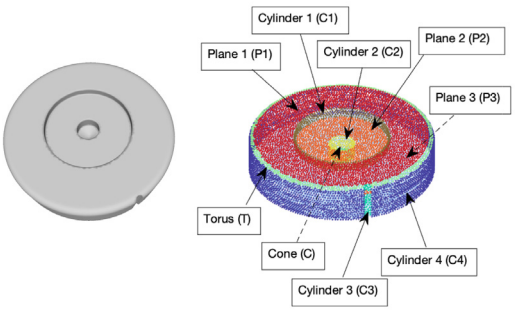
6. Experimental results

In this section, we show the results obtained with our method. Specifically, Section 6.1.1 presents some point clouds of mechanical objects containing only simple geometric primitives, while Section 6.1.2 provides some examples with complex primitives. In Section 6.1.3, we exhibit some noisy point clouds to demonstrate the robustness of our pipeline. In all cases, whenever possible, we show the maximal primitives of the selected point cloud. Finally, a qualitative and quantitative comparative analysis is provided in Section 6.2.

6.1. Overall analysis

6.1.1. Simple geometric primitives

As a sanity check, we first apply our method to two models from [42], which were selected because containing all simple geometric primitives – plane, sphere, cylinder, cone and torus – and because of the availability of ground truth. A first example is provided in Fig. 3. The point cloud, corresponding to the set of vertices of the original triangle mesh, is decomposed in 8 surface segments: 5 cylinders, 2 planes and 1 sphere. The high accuracy of our method is proved by comparing the parameters from the HT with those in the database. In the second example, presented in Fig. 4, the corresponding point cloud is subdivided into 9 surface primitives: 4 cylinders, 1 torus, 3 axis-aligned planes and 1 cone. Again, we are able to recognize all primitives up to a small error in the parameters with respect to those provided in the dataset.



(a) (b)

equation	code	ground truth	HT parameters
$x^2+y^2=r^2$	C1	$r=15.01$	$r=15.01$
	C2	$r=3.97$	$r=3.97$
	C3	$r=1.28$	$r=1.26$
	C4	$r=29.25$	$r=29.25$
$z=k$	P1	$k=12.00$	$k=12.00$
	P2	$k=9.08$	$k=9.08$
	P3	$k=0.00$	$k=0.00$
$(R-\sqrt{x^2+y^2})^2+z^2-r^2=0$	T	$R=27.25$	$R=27.25$
		$r=2.00$	$r=2.00$
$x^2+y^2+a(z-b)^2=0$	C	$a=-2.77$	$a=-2.81$
		$b=2.19$	$b=2.22$

(c)

Fig. 4. In (a) a mechanical CAD model from the benchmark in [42]; in (b) the vertices of its triangle mesh decomposed into 9 surface segments; in (c), for each primitive, the HT parameters are compared with those provided by the database.

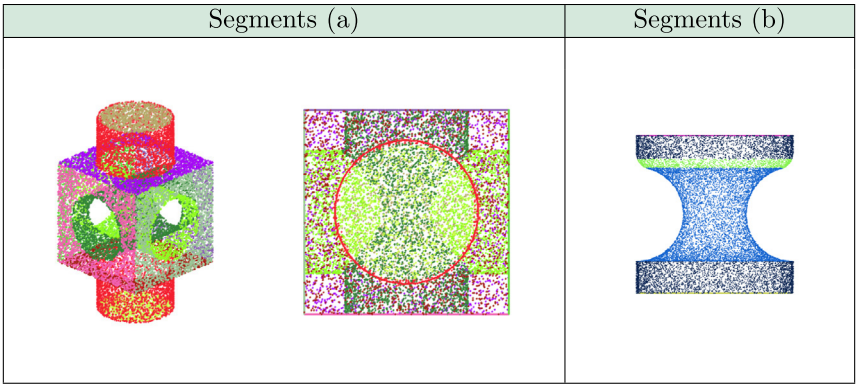


Fig. 5. Recognition of CAD point clouds containing only simple geometric primitives. The identification of maximal segments does not require, in these cases, the application of any clustering algorithm.

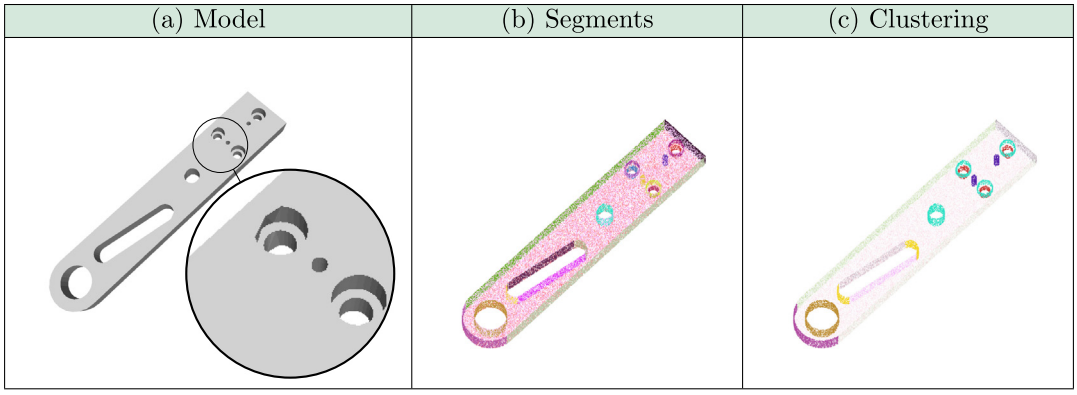


Fig. 6. A linkage arm. In (a), the original model is shown, together with a magnification revealing some imperfections. The segments identified by the HT are shown in (b) in different colours. (c) draws attention to cylinders, among which we can recognize segments lying on the same primitive, up to a translation.

Figs. 5(a-b) show point clouds that can be segmented into complete geometric primitives without the need for a further step of aggregation. The first example, shown in Fig. 5(a), consists of 11 segments – 3 cylinders and 8 planes – and it highlights the robustness of our method when it comes to detecting intersecting cylinders, here arranged similarly to a Steinmetz solid. Complete geometric primitives, each of which is represented by a specific colour, are automatically detected. Another point cloud, displayed in Fig. 5(b), contains 5 segments: 2 tori, 1 cylinder and 2 axis-aligned planes. As for the previous case, the HT-based recognition successfully detects all the primitives, even those made up of non-adjacent parts.

Figs. 6–9 show point clouds wherein the segments produced by the HT approach are post-processed by hierarchical clustering. In Fig. 6(b), the input point cloud is first segmented into 20 surface primitives via the HT-based recognition algorithm (cylinders and planes), which are then associated thanks to the comparison of the geometric descriptors that characterize them uniquely. As expected, no pair of primitives lying on the same surface is found. Despite the presence of imperfections in the original model (see Fig. 6(a)), we can correctly identify repeating primitives, here in the form of circular cylinders of equal radii. Fig. 6(c) highlights the similarities identified by associating cylinders: 4 in light blue, 3 in red, 2 in purple and 2 in yellow.

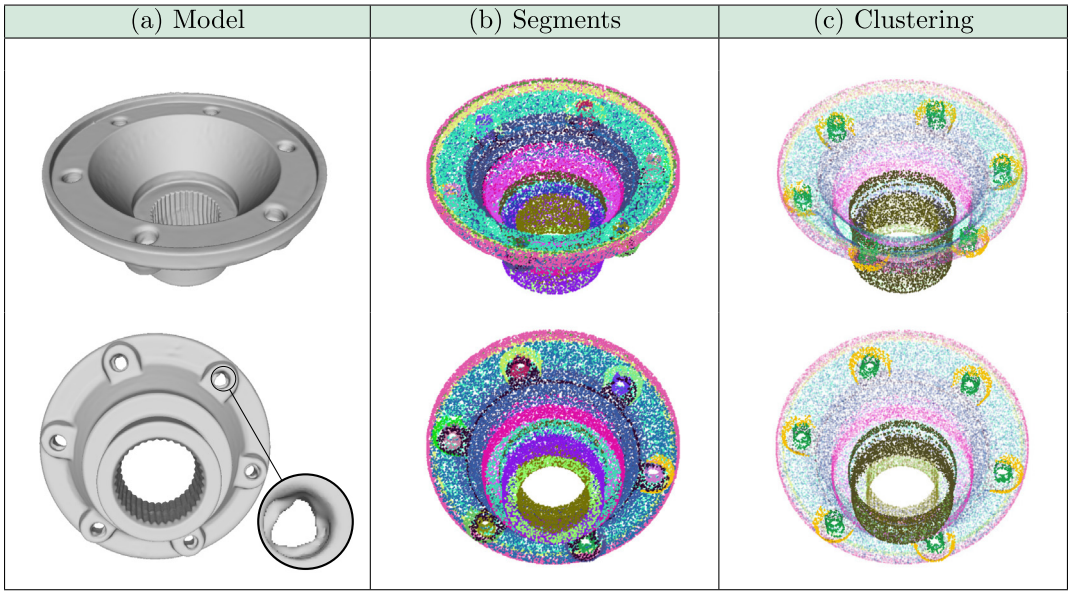


Fig. 7. A carter. In (a), the original model is shown. The surface segments found by the HT approach are depicted in (b), while (c) shows the result of primitive association, when one is interested in identifying the same primitive up to a translational transformation. Different rows correspond to different points of view.

Fig. 7 increases the difficulty by considering a point cloud containing a higher number of segments, some of which are low-quality as the small holes highlighted in Fig. 7(a). In this example, the geometric primitives identified by the HT algorithm are cylinders, cones, and planes. The result is a segmentation in 28 surface segments, see Fig. 7(b). Note that the central hole presents some grooves, i.e., a surface detail that was not present as a geometric feature in the original CAD model; therefore, we recognize it as a cylinder and the HT is able to ignore the shallow grooves. A final application of clustering makes it possible to identify repeating primitives, up to translations. Fig. 7(c) illustrates the similarity between 6 cylindrical holes (in green) and between other 6 cylindrical segments (in yellow).

Another example of gear is shown in Fig. 8(a). Here, the total number of extracted geometric primitives is 68: 19 cylinders; 2 cones; 47 planes, 5 of which are axis-aligned. The result is presented in Fig. 8(b). As highlighted by the original model, in this prototypical version of the NuGear component, cylinders are roughly approximated by a series of planar primitives; in this specific case, we fit a cylinder instead of many planes, since the former can describe a much larger area without significantly increasing the error. Clustering identifies here a similarity between the 12 cylindrical holes – up to translations – and between 2 external cylinders, while the surface segments identified by non-axis-aligned planes are not grouped in pairs; this is because the tooth inclination prevents any alignment. Fig. 8(c) shows this result, colouring the holes in black and the external cylinders in yellow.

Finally, Fig. 9(a) shows another mechanical part. The HT-based decomposition of the input point cloud consists of 50 surface segments, all of which are tori, cylinders, and planes, see Fig. 9(b). The clustering method can identify the similarity between 8 red cylindrical holes and between 2 blue cylindrical segments, up to translations — see Fig. 9(c). Finally, 2 tori are aggregated, because they lie on the same surface primitive up to rototranslations.

Table 2 summarizes the characteristics of each point cloud processed in this section and the mean fitting error for all the simple primitives recognized on it.

Table 2
Statistics of the MFES for all models of Section 6.1.1. Being the MFE normalized by definition, we can conclude that the maximum error for the fitting of the simple primitives is 4.48%, which corresponds to the noisy holes in the carter of Fig. 7.

Model	# points	# segs	min(E_i)	mean(E_i)	max(E_i)
Fig. 3	15,216	8	0.0006	0.0021	0.0046
Fig. 4	15,022	9	0.0008	0.0029	0.0051
Fig. 5(a)	25,000	11	0.0009	0.0024	0.0056
Fig. 5(b)	25,000	5	0.0004	0.0031	0.0059
Fig. 6(b)	50,000	20	0.0004	0.0098	0.0300
Fig. 7(b)	50,000	28	0.0013	0.0107	0.0448
Fig. 8(b)	50,000	68	0.0006	0.0053	0.0178
Fig. 9(b)	50,000	50	0.0008	0.0035	0.0057

6.1.2. Complex geometric primitives

As anticipated, our method can recognize other primitives in addition to the simple ones shown in Section 6.1.1. Here we show some of the complex primitives identified in our experiments.

The first example – shown in Fig. 10(a) – contains an ellipsoid, which is easily recognized by our method at the price of an additional parameter in the parameter space; additionally, the point cloud can be partitioned into 4 planes, 1 torus and 1 cylinder. In the point cloud from Fig. 10(b), we are able to identify correctly the yellow segment as a surface of revolution from Table 1(c); the remaining points are segmented into 2 cylinders and 2 planes. In the point cloud shown in Fig. 10(c), we recognize the gold part as a generalized cone, while the blue and the green segments are fitted with generalized cylinders: all of the three have the same directrix, a 5—convexity curve. This last point cloud has been segmented into 7 clusters.

Fig. 11(a) displays a mechanical part which can be accurately described by combining a portion of a helical surface, as introduced in Table 1(d), with a pair of planes and a pair of convex combinations of helices, see Table 1(e). The result is a segmentation of the point cloud into 6 primitives, Fig. 11(b). Two of them are then grouped by the clustering technique since the helical strips have the same equation up to a translation, as shown in Fig. 11(c).

Table 3 provides the main characteristics of each point cloud processed in this section and the mean fitting error for all the simple and complex primitives recognized on it.

6.1.3. Robustness of the pipeline

The use of the HT naturally leads to a robust method for the recognition of mathematical surfaces, as suggested in the

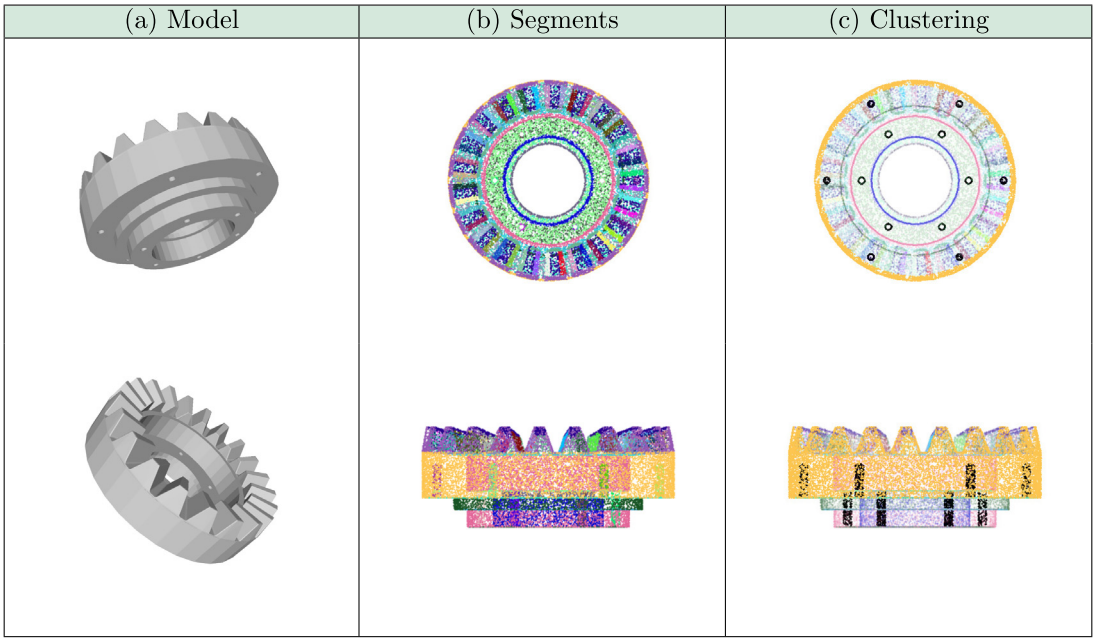


Fig. 8. A prototype of the NuGear component, courtesy of STAM S.r.l. (Genoa, Italy). The original model is shown in (a). The decomposition in clusters of points produced by the HT approach is given in (b). The output of the additional clustering association procedure is shown in (c), which highlights the similarity between 12 cylindrical holes (in black) and between two cylinders (in yellow). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

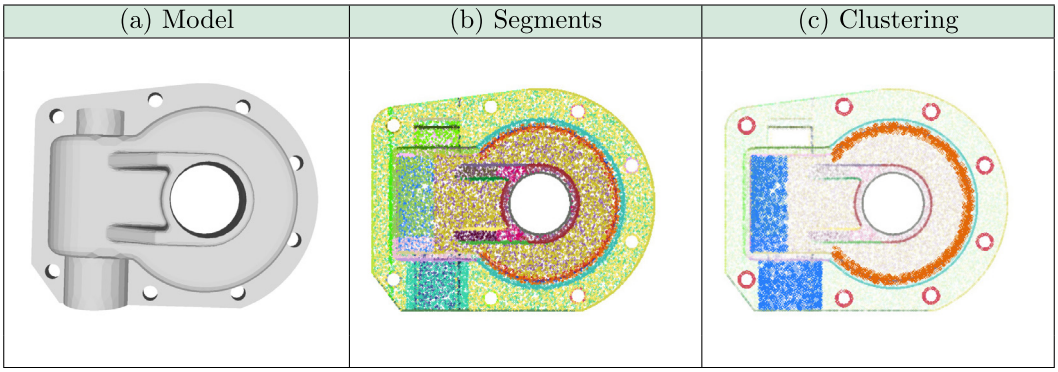


Fig. 9. A mechanical part. In (a) the original model is shown, while in (b) the decomposition of the corresponding point cloud into segments produced by the HT. In (c) the result of the clustering procedure: 8 cylindrical holes, in red, have a high similarity, up to translations; the same applies for 2 cylindrical segments, in blue; 2 tori, in orange, identify the same primitive, up to a rototranslation.

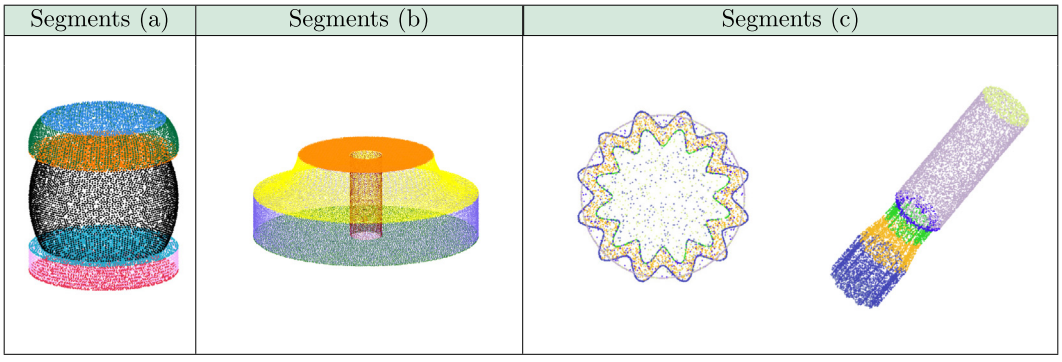


Fig. 10. Recognition of complex geometric primitives in CAD point clouds. Their identification does not require, in these cases, the application of any clustering algorithm.

examples of Figs. 6 and 7 – which were characterized by spurious parts.

In the point cloud of Fig. 12, the HT recognition correctly identifies the cylinder that fits the central part, without being

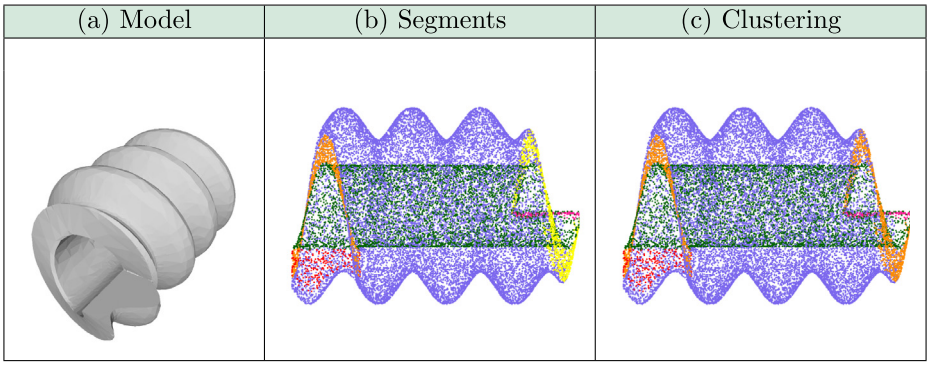


Fig. 11. A screw-like part. The original model, (a), is sampled. The surface primitives detected via HT are shown in (b) using different colours: a helical surface (in purple), two planes (in red and magenta), and two helical strips (in orange and yellow). Although no pair of them lies on the same parametrized surface, the 2 helical strips have the same equation up to a translation, as shown in (c). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

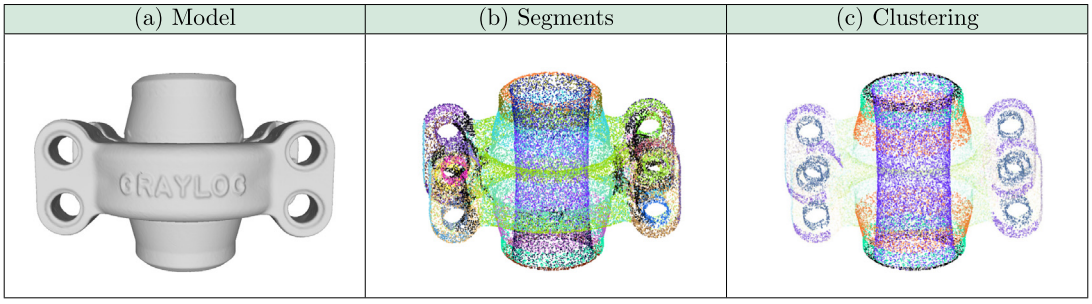


Fig. 12. A clamp connector. In (a) the original model. In (b) the decomposition of the corresponding point cloud into 38 segments provided by the HT procedure. In (c), the final grouping obtained by clustering, consisting of 6 clusters of primitives (here, singletons of segments are transparent).

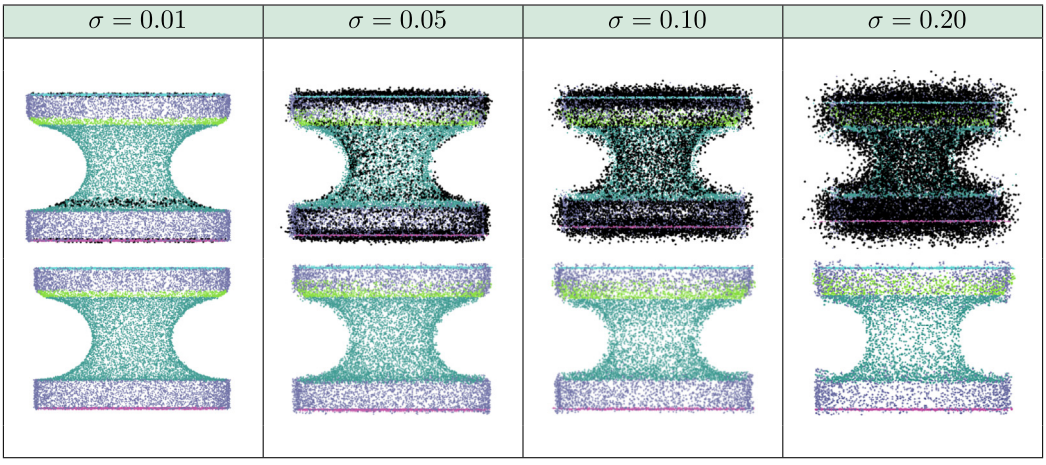


Fig. 13. The point cloud in Fig. 5(b) is perturbed by adding zero-mean Gaussian noise of standard deviation: 0.01, 0.05, 0.10 and 0.20. The first row superimposes the points identified as noise (in black) to the final segments found by our method; the second row depicts the points that fit the primitives found and provides a denoised segmentation. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 3
Statistics of the MFEs for all models of Section 6.1.2. Being the MFE normalized by definition, we can conclude that the maximum error for the fitting of the simple primitives is 1, 54%, which corresponds to the helical surface of Fig. 11.

Model	# points	# segs	min(E_i)	mean(E_i)	max(E_i)
Fig. 10(a)	25,524	7	0.0009	0.0042	0.0063
Fig. 10(b)	67,777	5	0.0006	0.0031	0.0071
Fig. 10(c)	25,000	7	0.0020	0.0053	0.0081
Fig. 11(b)	25,000	6	0.0033	0.0086	0.0154

negatively influenced by the letters in relief — see the original model in Fig. 12(a). The point cloud is decomposed into 38 segments: 23 cylinders with different axes, 10 planes, 4 cones, and 1 torus that automatically identifies the top and bottom of the cylinder with the "GRAYLOC" inscription (see Fig. 12(b)). The application of the hierarchical clustering technique allows us to group together: 8 grey cylindrical holes (up to rototranslations); 8 purple cylindrical segments; 2 aquamarine circular cylinders; 3 violet circular cylinders; 2 orange circular cones (up to a reflection); 2 black cones (up to a reflection). Moreover, the small imperfections of the manufacture on the central part of the body (recognized by vertical cones, cylinders and tori at the top and bottom) and on the lateral holes do not prevent the clustering

Table 4
Parameter comparison between the original point cloud from Fig. 5(b) and the perturbed versions from Fig. 13.

Segment	Original	$\sigma = 0.01$	$\sigma = 0.05$	$\sigma = 0.10$	$\sigma = 0.20$
Plane 1	$k = -1.28$	$k = -1.28$	$k = -1.29$	$k = -1.28$	$k = -1.31$
Plane 2	$k = 1.28$	$k = 1.28$	$k = 1.28$	$k = 1.28$	$k = 1.26$
Cylinder	$r = 1.80$	$r = 1.79$	$r = 1.78$	$r = 1.79$	$r = 1.78$
Torus 1	$R = 1.49$ $r = 0.72$	$R = 1.49$ $r = 0.73$	$R = 1.47$ $r = 0.70$	$R = 1.48$ $r = 0.67$	$R = 1.56$ $r = 0.74$
Torus 2	$R = 1.05$ $r = 0.79$	$R = 1.09$ $r = 0.78$	$R = 1.02$ $r = 0.78$	$R = 1.17$ $r = 0.74$	$R = 1.13$ $r = 0.80$

technique from appropriately associating the corresponding segments, correctly dealing with rotations and reflections. However not everything is recognized: the black dots in Fig. 12(b) correspond to points that are not fitted by any of the geometric primitives at our disposal, as they originate from irregular elements that act as a connection between better-defined segments. We label such points as "unsegmented" because of the high mean fitting error.

Further proof of the robustness of our method applied to raw data is presented in Fig. 13 and quantitatively analysed in Table 4. In this example, we perturb the point cloud of Fig. 5(b) by adding zero-mean Gaussian noise of standard deviation: 0.01, 0.05, 0.10 and 0.20. The first row shows the points classified as noise (in black) and the segments found by our method in the same image, for each level of noise. The second row focuses only on the points that fit the identified primitives, thus providing a denoised segmentation. The robustness to noise is quantitatively studied in Table 4: the parameters obtained in the original point cloud are there compared with those found in the perturbed point clouds.

6.2. Comparative analysis

We here present two comparisons with alternative methods, all of which rely on estimating the normal vectors at the given input points.

The first analysis, proposed in Fig. 14, is merely visual. Due to the lack of freely-downloadable implementations for some methods, it is indeed not possible to present this comparison other than qualitatively. The analysis consists of a comparison of our method with the RANSAC-based method introduced in [11], the patch aggregation approach in [15] and two recent deep learning architectures: ParSeNet [24] and HPNet [25]. In this comparison, we have focused on models that solely present simple primitives, to show that even on these our approach gives a great performance; on the other hand, the capability to handle more complex primitives is undoubtedly an added value of our method. Similarly to [15], we use different colours to represent different primitive typologies: red for planes, green for cylinders, blue for cones, black for tori, pink for (open and closed) B-splines and yellow for unsegmented parts. For a simple model like the block of Fig. 14(a), all methods provide decent decompositions, although RANSAC misclassifies some primitives while the deep learning frameworks run into problems because of an inaccurate estimation of the surface normals: this assertion is supported by the location of misclassified points, which are largely limited to the sharp edges. In this regard, it is worth noting that – considering that the networks were trained on clean data – the more accurate the normals, the more precise the final segmentation. In our case, the use of a voting procedure allows us to withstand possible perturbations. For the remaining more complex models, our method outperforms the competitors. In Fig. 14(b), our approach is the only capable of correctly identifying portions of tori, misclassified by Le and Duan [15] and partly unsegmented by RANSAC; ParSeNet and HPNet are able to reveal the presence of tori, but the resulting segmentation is unreliable along sharp edges — again, mostly because of a deficiency in the normal estimates. Figs. 14(c-d) show a RANSAC tendency to oversegment and misclassify complex models. While Le and Duan [15] obtain considerably improved results, their algorithm misses a thin cylinder (see Fig. 14(c)) and all the tori in both models. Being these two objects acquired by low-quality scanners, the corresponding point clouds (and meshes) are affected by point cloud artefacts which, in turn, lead to an even greater error in the normal estimates. The resulting segmentations are unreliable and mostly associated with spline segments. As previously mentioned, note that the two architectures were trained on the ABC dataset, which does not take into account the presence of noise or other types of perturbation in the data. Due to a lack of a sufficiently rich benchmark of scanned CAD objects, however, new training is not currently possible.

To offer a quantitative analysis of the robustness of our pipeline, we compare its performance with that of three state-of-the-art methods whose implementation was made freely available by the authors: a direct method approach on a primitive-growing (PG) framework [18], adapted to handle point clouds as described in [19], and the two learning-based methods from the previous comparison: ParSeNet (PN) and HPNet (HN). We conduct the study over the whole Fit4CAD benchmark [19], which contains CAD point clouds defined by simple geometric primitives. In addition, Fit4CAD contains a selection of meaningful and validated models from [42], converted from meshes to point clouds. Fig. 15 summarizes the performance of each method over the test set with respect to the following classification measures: Sørensen-Dice index (DSC), Positive Predicted Value (PPV), True Positive Rate (TPR), Negative Predicted Value (NPV), True Negative Rate (TNR) and accuracy (ACC). Given a point cloud, the benchmark defines these measures by comparing each point cloud segment in the ground truth to the most overlapping segment returned by a segmentation method, and then by averaging over all segments contained in that point cloud. Note that, given a point cloud segment in the ground truth, the points inside that segment are the positives while the points outside that segment are the negatives. We arrive at the following conclusions:

- Learning-based methods show remarkably high TNRs — said otherwise, the points they predict as being negative are almost always true negatives. On the other hand, they are more penalized by NPV: while it is ParSeNet that exhibits the highest variability and the lowest quartiles, both methods are seriously affected by outliers — with some point clouds having this score below 0.2. A low NPV means that quite a few points predicted as negative are false negatives (e.g., common in heavily oversegmented models). In this case, these methods have been penalized by the low density of the point clouds, as well as from the possible presence of point cloud artefacts (e.g., missing data).
- Our approach performs significantly better than the competitors in terms of TPR, i.e., it has similar proportions of correct (positive) predictions among positives. When it comes to the PPV, differences are more modest.

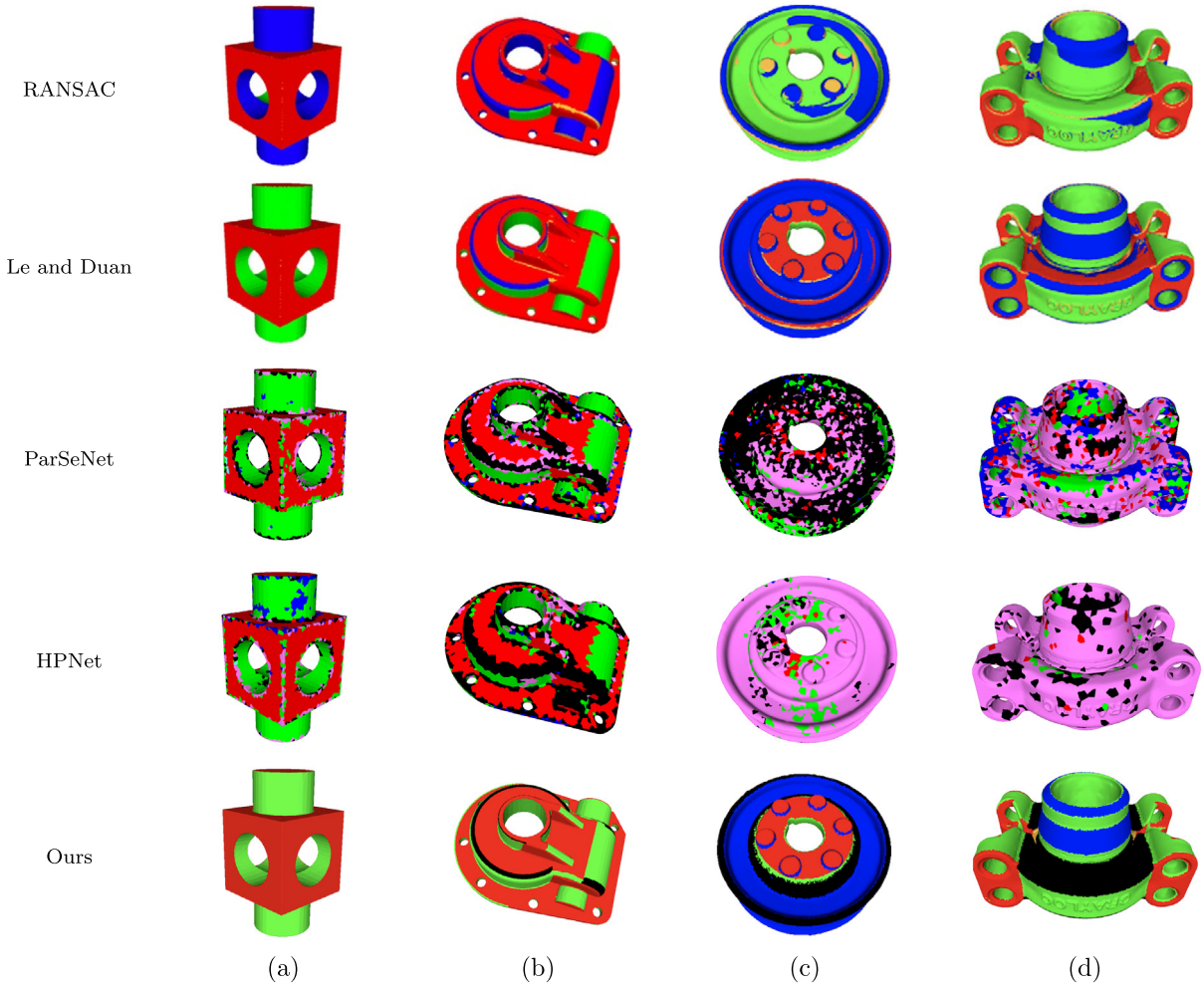


Fig. 14. Primitive type recognition: a comparison between our approach, a RANSAC-based segmentation [11], and the method in [15]. Different colours correspond to different primitive types: planes (red), cylinders (green), cones (blue), tori (black), splines (pink), unsegmented (yellow). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

- In terms of accuracy, the four methods reach high scores, with the direct methods being the less prone to outliers: however, this metric is not completely reliable as this is a naturally unbalanced binary classification problem. A more reliable measure is provided by the Sørensen-Dice index (DSC). Our method visibly outperforms the competitors in terms of the DSC. Intuitively, the higher the DSC, the more accurate segments returned by a segmentation method are — with respect to the most overlapping segment in the ground truth; to put it differently, DSC penalizes greatly both under- and oversegmentation.

When it comes to execution time, our Hough-based method and the primitive-growing approach were run on a desktop PC equipped with an Intel Core i9 processor (at 3.6 GHz) and a Windows 10 operating system. The average execution time, per model, are 286.0 s for the PG-method and 50.7 s for our pipeline. ParSeNet and HPNet were run on Google Colab pro equipped with NVIDIA-SMI 460.32.03 and CUDA 11.2. Regarding the average execution times, we have 5 seconds for ParSeNet and 257.1 s for HPNet (when the preprocessing of normals is applied).

7. Conclusions

To address the problem of recognition of simple and complex primitives in a point cloud, we have opportunely extended the family of geometric primitives to which the HT technique can be applied. The HT is naturally robust to noise and outliers and recognizes multiple instances of the same primitive. Thanks to an opportune preprocessing of the point cloud and its sub-parts, we are able to limit the number of parameters that are necessary to represent the primitive, thus reducing the computational complexity of the HT computation. In addition, the explicit extraction of position-independent primitive parameters and the use of a hierarchical clustering strategy permits us to identify maximal and compound primitives, thus reducing the output oversegmentation. Indeed, differently from spline-based primitives, our strategy is suited for primitive similarity reasoning and permits us to find maximal primitives.

Although learning methods have performed very well in recent years, when models present complex combinations of primitives (such as the examples in Figs. 14(c,d)), direct methods perform even better, and our method is very competitive. In addition, our method has been validated on a whole benchmark and, compared with the others, it turns out to be the best.

The use of geometric primitives whose mathematical representations require a large number of parameters – even in their standard forms – is currently the main limitation of our pipeline; while theoretically possible, this would indeed increase the execution time exponentially, making the algorithm inapplicable in practice. An instructive example, shown in Fig. 16, is given by point clouds that contain segments generated by spline surfaces: while our method cannot deal with spline surfaces, it does

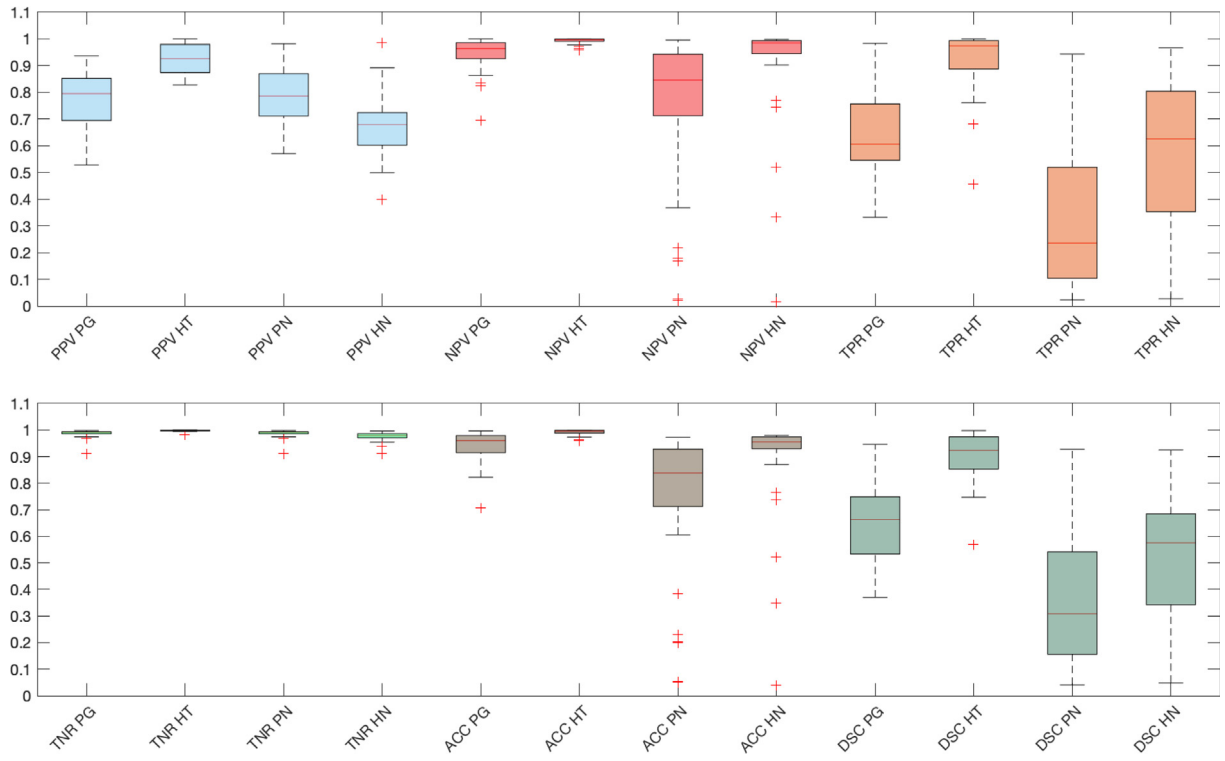


Fig. 15. Comparison of our approach (HT) with other three methods: a primitive growing approach (PG), ParSeNet (PN) and HPNet (HN). The analysis is performed over the Fit4CAD benchmark [19].

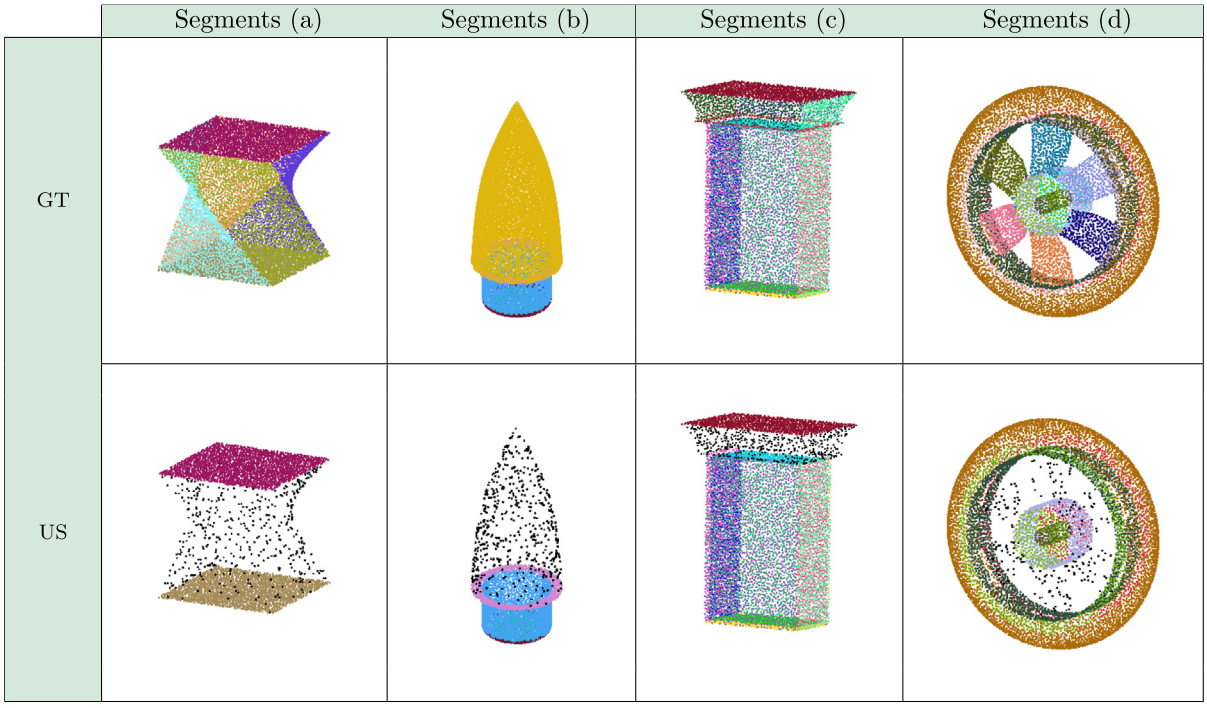


Fig. 16. Four models from the ABC dataset [42] containing spline segments: comparison between the ground truth (GT) segmentation and our recognition of simple geometric primitives (US). To enhance the visibility of the primitives correctly recognized, the second row displays the segments classified as splines with a lower point density.

manage to recognize that parts of a point cloud do not originate from any primitive at our disposal. The first row of Fig. 16 shows the ground truth segmentation, while the second row shows the segmentation we achieve: the black dots correspond to points that are not recognized as simple geometric primitives, and that are labelled as "unsegmented" because of the high mean fitting error; note that spline segments are undersampled for the sake of visibility, but this processing is realized only after segmenting the point clouds. On the contrary, methods that recognize splines tend to give any segment a label, as shown in Fig. 14.

As future work, we are thinking of other strategies to reduce the computational cost in the HT-based recognition step. Indeed, the parameter space can be further optimized by reducing its dimension through the use of a refinement strategy that discretizes the parameter space only in the relevant regions of interest. Another possible direction of investigation could concern the applicability of our method to non-orientable surfaces. Indeed we have only applied our method to objects with orientable surfaces since the datasets we have used exhibit only this type of surface. However, in theory, our approach could also apply to objects with non-orientable surfaces, as long as their equation exists in the literature and the system of equations is solvable with respect to the parameter (as in the case of Möbius strip). The challenge here would be to study a preprocessing technique to translate and/or rotate the point cloud into the standard form of that surface type. For instance, in the case of the Möbius strip, how to estimate the symmetry axis in order to place the point cloud so that we can fit it with a primitive in a standard form.

Regarding the automation of the process, we could take advantage of a pre-classification of the points in the input point cloud using a curvature-based characterization, for instance, the local surface variation proposed in [46] or the shape index and curvedness as used in [18] for ISO GPS segmentation. Once points are aggregated by their type, we could further split or aggregate these regions using our fitting method and possibly considering only a suitable subset of primitives, i.e., the primitives that are compatible with the recognized type of points, e.g., planar, cylindrical, spherical, etc.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work has been developed in the CNR research activities DIT.AD004.100, DIT.AD021.080.001 and DIT.AD021.125.

The authors thank the VVS shape repository by the AIM@SHAPE consortium (http://visionair.ge.imati.cnr.it) and Dr. Alexander Leutgeb from RISC Software GmbH (Linz, Austria) for the models used in this paper. The authors also thank Dr. Andrea Ranieri for helping with the Google Colab platform used in the comparative analysis.

References

[1] Poux F, Mattes C, Selman Z, Kobbelt L. Automatic region-growing system for the segmentation of large point clouds. Autom Constr 2022;138:104250. http://dx.doi.org/10.1016/j.autcon.2022.104250, https://www.sciencedirect.com/science/article/pii/S0926580522001236.

[2] Li Y, Wu X, Chrysathou Y, Sharf A, Cohen-Or D, Mitra NJ. GlobFit: Consistently Fitting Primitives by Discovering Global Relations. ACM Trans Graph 2011;30(4). http://dx.doi.org/10.1145/2010324.1964947.

[3] Monszpart A, Mellado N, Brostow GJ, Mitra NJ. Rapter: Rebuilding man-made scenes with regular arrangements of planes. ACM Trans Graph 2015;34(4). http://dx.doi.org/10.1145/2766995.

[4] Lupinetti K, Pernot J-P, Monti M, Giannini F. Content-based CAD assembly model retrieval: Survey and future challenges. Comput Aided Des 2019;113:62–81. http://dx.doi.org/10.1016/j.cad.2019.03.005.

[5] Hough PVC. Method and means for recognizing complex patterns. US Patent 3, 069 654 1962. https://www.osti.gov/biblio/4746348.

[6] Ballard DH. Generalizing the Hough transform to detect arbitrary shapes. Pattern Recognit 1981;13(2):111–22. http://dx.doi.org/10.1016/0031-3203(81)90009-1.

[7] Woodford OJ, Pham M, Maki A, Perbet F, Stenger B. Demisting the Hough Transform for 3D Shape Recognition and Registration. Int J Comput Vis 2014;106(3):332–41. http://dx.doi.org/10.1007/s11263-013-0623-2.

[8] Beltrametti MC, Robbiano L. An algebraic approach to Hough transforms. J Algebra 2012;37:669–81. http://dx.doi.org/10.1016/j.jalgebra.2012.09.012.

[9] Raffo A, Romanengo C, Falcidieno B, Biasotti S. Fitting and recognition of geometric primitives in segmented 3D point clouds using a localized voting procedure. Comput Aided Geom Design 97. http://dx.doi.org/10.1016/j.cagd.2022.102123.

[10] Kaiser A, Ybanez Zepeda JA, Boubekeur T. A Survey of Simple Geometric Primitives Detection Methods for Captured 3D Data. Comput Graph Forum 2019;38(1):167–96. http://dx.doi.org/10.1111/cgf.13451.

[11] Schnabel R, Wahl R, Klein R. Efficient RANSAC for Point-Cloud Shape Detection. Comput Graph Forum 2007;26(2):214–26. http://dx.doi.org/10.1111/j.1467-8659.2007.01016.x.

[12] Limberger FA, Oliveira MM. Real-time detection of planar regions in unorganized point clouds. Pattern Recognit 2015;48(6):2043–53. http://dx.doi.org/10.1016/j.patcog.2014.12.020.

[13] Attene M, Patanè G. Hierarchical structure recovery of point-sampled surfaces. Comput Graph Forum 2010;29(6):1905–20. http://dx.doi.org/10.1111/j.1467-8659.2010.01658.x.

[14] Yan D-M, Wang W, Liu Y, Yang Z. Variational mesh segmentation via quadric surface fitting. Comput Aided Des 2012;44(11):1072–82. http://dx.doi.org/10.1016/j.cad.2012.04.005.

[15] Le T, Duan Y. A primitive-based 3D segmentation algorithm for mechanical CAD models. Comput Aided Geom Des 2017;52-53:231–46. http://dx.doi.org/10.1016/j.cagd.2017.02.009, Geometric Modeling and Processing 2017.

[16] Li L, Sung M, Dubrovina A, Yi L, Guibas LJ. Supervised Fitting of Geometric Primitives to 3D Point Clouds. In: Proceedings – IEEE Computer society conference on computer vision and pattern recognition. 2019, p. 2652–60. http://dx.doi.org/10.1109/CVPR.2019.00276.

[17] Uy MA, Chang Y-Y, Sung M, Goel P, Lambourne JG, Birdal T, Guibas LJ. Point2cyl: Reverse engineering 3D objects from point clouds to extrusion cylinders. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022, p. 11850–60.

[18] Qie Y, Qiao L, Anwer N. Enhanced invariance class partitioning using discrete curvatures and conformal geometry. Comput Aided Des 133. http://dx.doi.org/10.1016/j.cad.2020.102985.

[19] Romanengo C, Raffo A, Qie Y, Anwer N, Falcidieno B. Fit4CAD: A point cloud benchmark for fitting simple geometric primitives in CAD objects. Comput Graphics 2022;102:133–43. http://dx.doi.org/10.1016/j.cag.2021.09.013.

[20] Markovic V, Jakovljevic Z, Budak I. Automatic recognition of cylinders and planes from unstructured point clouds. Vis Comput 2021;1–24.

[21] Birdal T, Busam B, Navab N, Ilic S, Sturm P. Generic primitive detection in point clouds using novel minimal quadric fits. IEEE Trans Pattern Anal Mach Intell 2020;42(6):1333–47. http://dx.doi.org/10.1109/TPAMI.2019.2900309.

[22] Liu C, Hu W. Real-time geometric fitting and pose estimation for surface of revolution. Pattern Recognit 2019;85:90–108. http://dx.doi.org/10.1016/j.patcog.2018.08.002.

[23] Bergamasco F, Pistellato M, Albarelli A, Torsello A. Cylinders extraction in non-oriented point clouds as a clustering problem. Pattern Recognit 2020;107:107443. http://dx.doi.org/10.1016/j.patcog.2020.107443, https://www.sciencedirect.com/science/article/pii/S0031320320302466.

[24] Sharma G, Liu D, Maji S, Kalogerakis E, Chaudhuri S, Měch R. ParSeNet: A parametric surface fitting network for 3D point clouds. In: Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August (2020) 23–28, Proceedings, Part VII. Berlin, Heidelberg: Springer-Verlag; 2020, p. 261–76. http://dx.doi.org/10.1007/978-3-030-58571-6_16.

[25] Yan S, Yang Z, Ma C, Huang H, Vouga E, Huang Q. HPNet: Deep Primitive Segmentation Using Hybrid Representations. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2021, p. 2753–62. http://dx.doi.org/10.1109/ICCV48922.2021.00275.

[26] Sharma G, Dash B, RoyChowdhury A, Gadelha M, Loizou M, Cao L, Wang R, Learned-Miller EG, Maji S, Kalogerakis E. PriFit: Learning to fit primitives improves few shot point cloud segmentation. Comput Graph Forum http://dx.doi.org/10.1111/cgf.14601.

[27] Borrmann D, Elseberg J, Lingemann K, Nüchter A. The 3D Hough transform for plane detection in point clouds: A review and a new accumulator design. 3D Res 2011;2(2). http://dx.doi.org/10.1007/3DRes.02(2011)3.

[28] Fernandes LA, Oliveira MM. A general framework for subspace detection in unordered multidimensional data. Pattern Recognit 2012;45(9):3566–79. http://dx.doi.org/10.1016/j.patcog.2012.02.033.

[29] Camurri M, Vezzani R, Cucchiara R. 3D Hough Transform for Sphere Recognition on Point Clouds. Mach Vis Appl 2014;25(7):1877–91.

[30] Rabbani Shah T, van den Heuvel F. Efficient Hough transform for automatic detection of cylinders in point clouds. In: Vosselman G, Brenner C, editors. ISPRS Laser Scanning 2005, ISPRS Working Groups. 2005, p. 60–5.
[31] Beltrametti MC, Sendra JR, Sendra J, Torrente M. Moore–Penrose approach in the Hough transform framework. Appl Math Comput 2020;375:125083. http://dx.doi.org/10.1016/j.amc.2020.125083.
[32] Romanengo C, Raffo A, Biasotti S, Falcidieno B, Fotis V, Romanelis I, Psatha E, Moustakas K, Sipiran I, Nguyen Q-T, Chu C-B, Nguyen-Ngoc K-N, Vo D-K, To T-A, Nguyen N-T, Le-Pham N-Q, Nguyen H-D, Tran M-T, Qie Y, Anwer N. SHREC 2022: Fitting and recognition of simple geometric primitives on point clouds. Comput. Graph. 2022;107:32–49. http://dx.doi.org/10.1016/j.cag.2022.07.004, https://www.sciencedirect.com/science/article/pii/S0097849322001224.
[33] Shikin EV. Handbook and atlas of curves. CRC Press; 1995.
[34] Hoppe H, DeRose T, Duchamp T, McDonald J, Stuetzle W. Surface reconstruction from unorganized points. In: Proceedings of the 19th Annual conference on computer graphics and interactive techniques, SIGGRAPH '92, Association for Computing Machinery, New York, NY, USA. 1992, p. 71–8. http://dx.doi.org/10.1145/133994.134011.
[35] Romanengo C, Brunetto E, Biasotti S, Catalano CE, Falcidieno B. Recognition, modelling and interactive manipulation of motifs or symbols represented by a composition of curves. In: Biasotti S, Pintus R, Berretti S, editors. Italian chapter conference 2020 - Smart tools and apps in computer graphics, STAG 2020, Virtual Event, Italy, November (2020) 12-13. Eurographics Association; 2020, p. 27–35. http://dx.doi.org/10.2312/stag.20201237.
[36] Penrose R. On best approximate solutions of linear matrix equations. Math Proc Camb Phil Soc 1956;52(1):17–9.
[37] Edelsbrunner, Letscher, Zomorodian. Topological persistence and simplification. Discrete Comput Geom 2002;28(4):511–33. http://dx.doi.org/10.1007/s00454-002-2885-2.
[38] Doraiswamy H, Shivashankar N, Natarajan V, Wang Y. Topological saliency. Comput. Graph. 2013;37(7):787–99. http://dx.doi.org/10.1016/j.cag.2013.04.009.
[39] Biasotti S, Cerri A, Pittaluga S, Sobrero D, Spagnuolo M. Tracking the evolution of rainfall precipitation fields using persistent maxima. In: Proceedings of the conference on smart tools and applications in computer graphics. 2016, p. 29–37. http://dx.doi.org/10.2312/stag.20161361.
[40] Friedman JH, Bentley JL, Finkel RA. An algorithm for finding best matches in logarithmic expected time. ACM Trans Math Software 1977;3(3):209–26. http://dx.doi.org/10.1145/355744.355745.
[41] Ester M, Kriegel HP, Sander J, Xu X. A density-based algorithm for discovering clusters in large spatial databases with noise. In: 2nd Int. Conf. Knowledge discovery and data mining. AAAI Press; 1996, p. 226–31.
[42] Koch S, Matveev A, Jiang Z, Williams F, Artemov A, Burnaev E, Alexa M, Zorin D, Panozzo D. ABC: A big CAD model dataset for geometric deep learning. In: The IEEE Conference on computer vision and pattern recognition. CVPR, 2019, http://dx.doi.org/10.1109/CVPR.2019.00983.
[43] Li H, Lavin MA, Le Master RJ. Fast Hough transform: A hierarchical approach. Comput Vis Graph Image Process 1986;36(2):139–61. http://dx.doi.org/10.1016/0734-189X(86)90073-3.
[44] Day W, Edelsbrunner H. Efficient algorithms for agglomerative hierarchical clustering methods. J Classification 1984;1(1):7–24. http://dx.doi.org/10.1007/BF01890115.
[45] Defays D. An efficient algorithm for a complete link method. Comput J 1977;20(4):364–6. http://dx.doi.org/10.1093/comjnl/20.4.364.
[46] Pauly M, Gross M, Kobbelt LP. Efficient simplification of point-sampled surfaces. IEEE Vis 2002;2002:163–70. http://dx.doi.org/10.1109/VISUAL.2002.1183771.